

FFT

```

#include <bits/stdc++.h>
using namespace std;
using cd = complex<double>;
const double PI = acos(-1);

// Fast Fourier Transform (Cooley-Tukey algorithm)
void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    // Bit-reversal permutation
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    // Iterative FFT
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1); // root of unity angle
        cd wlen(cos(ang), sin(ang)); // principal root of unity
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j];
                cd v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen; // next power of root
            }
        }
    }

    // Normalize if inverse FFT
    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

// Multiply two polynomials using FFT
vector<long long> multiply(const vector<long long> & a, const vector<long long> &
b) {
    if (a.empty() || b.empty()) return {};

```

```

vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
int n = 1;
while (n < a.size() + b.size())
    n <= 1;
fa.resize(n);
fb.resize(n);

// Perform FFT on both vectors
fft(fa, false);
fft(fb, false);

// Point-wise multiplication
for (int i = 0; i < n; i++)
    fa[i] *= fb[i];

// Inverse FFT to get result
fft(fa, true);

// Round real parts to nearest integer
vector<long long> result(n);
for (int i = 0; i < n; i++)
    result[i] = round(fa[i].real());

return result;
}

```

```

// Modular addition
int add(int a, int b) {
    return (a + b) % mod;
}

// Modular subtraction (ensures result is non-negative)
int sub(int a, int b) {
    return (a - b + mod) % mod;
}

// Modular exponentiation (a^b % mod)
ll poww(ll a, ll b) {
    ll ret = 1;
    while (b) {
        if (b & 1)
            ret = ret * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return ret;
}

// Fast Walsh-Hadamard Transform (FWHT)
// a: input array, inv: 0=forward, 1=inverse, f: type (0=AND, 1=OR, 2=XOR)

```

```

void fwht(vector<int> &a, int inv, int f) {
    int sz = a.size();
    for (int len = 1; 2 * len <= sz; len <= 1) {
        for (int i = 0; i < sz; i += 2 * len) {
            for (int j = 0; j < len; j++) {
                int x = a[i + j];
                int y = a[i + j + len];

                if (f == 0) { // AND convolution
                    if (!inv) {
                        // Forward transform for AND
                        a[i + j] = y;
                        a[i + j + len] = add(x, y);
                    } else {
                        // Inverse transform for AND
                        a[i + j] = sub(y, x);
                        a[i + j + len] = x;
                    }
                } else if (f == 1) { // OR convolution
                    if (!inv) {
                        // Forward transform for OR
                        a[i + j + len] = add(x, y);
                    } else {
                        // Inverse transform for OR
                        a[i + j + len] = sub(y, x);
                    }
                } else { // XOR convolution
                    // Same formula for forward and inverse, just normalize later
                    a[i + j] = add(x, y);
                    a[i + j + len] = sub(x, y);
                }
            }
        }
    }
}

// Multiply two bitmask arrays using FWHT
// f = 0 (AND), 1 (OR), 2 (XOR)
vector<int> mul(vector<int> a, vector<int> b, int f) {
    int sz = a.size();

    // Apply FWHT to both vectors
    fwht(a, 0, f);
    fwht(b, 0, f);

    // Pointwise multiplication
    vector<int> c(sz);
    for (int i = 0; i < sz; ++i)
        c[i] = 1ll * a[i] * b[i] % mod;

    // Inverse transform
    fwht(c, 1, f);

    // Only XOR requires normalization (by inverse of size)
}

```

```

if (f == 2) {
    int sz_inv = poww(sz, mod - 2); // Inverse using Fermat's Little Theorem
    for (int i = 0; i < sz; ++i)
        c[i] = 111 * c[i] * sz_inv % mod;
}

return c;
}

```

FFT with mod

```

#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for (int i = a; i < b; ++i)
#define sz(v) (int)(v).size()
typedef long long ll;
typedef complex<double> C;
typedef vector<int> vi;

const int MOD = 1e9 + 7; // Global modulus

// In-place Cooley-Tukey FFT
void fft(vector<C>& a) {
    int n = sz(a);
    int L = 31 - __builtin_clz(n);

    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i, k, 2 * k)
            rt[i] = R[i] = (i & 1 ? R[i / 2] * x : R[i / 2]);
    }

    vector<int> rev(n);
    rep(i, 0, n)
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i, 0, n)
        if (i < rev[i]) swap(a[i], a[rev[i]]);

    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            rep(j, 0, k) {
                auto x = (double*)&rt[j + k];
                auto y = (double*)&a[i + j + k];
                C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] * y[0]);
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
    }

```

```

    }
}

// Multiply polynomials a and b modulo MOD
vi convMod(const vi &a, const vi &b) {
    if (a.empty() || b.empty()) return {};

    int resultSize = sz(a) + sz(b) - 1;
    int B = 32 - __builtin_clz(resultSize);
    int n = 1 << B;
    int cut = int(sqrt(MOD));

    vector<C> L(n), R(n), outl(n), outs(n);
    rep(i, 0, sz(a))
        L[i] = C(a[i] / cut, a[i] % cut);
    rep(i, 0, sz(b))
        R[i] = C(b[i] / cut, b[i] % cut);

    fft(L), fft(R);

    rep(i, 0, n) {
        int j = -i & (n - 1);
        C a1 = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        C a2 = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / C(0, 1);
        outl[j] = a1;
        outs[j] = a2;
    }

    fft(outl), fft(outs);

    vi res(resultSize);
    rep(i, 0, resultSize) {
        ll av = ll(real(outl[i]) + 0.5);
        ll bv = ll(imag(outl[i]) + 0.5) + ll(real(outs[i]) + 0.5);
        ll cv = ll(imag(outs[i]) + 0.5);
        res[i] = ((av % MOD * cut + bv) % MOD * cut + cv) % MOD;
    }

    return res;
}

```

Polynomial power

```

// Raises a polynomial `poly` to the power `p` modulo  $x^{(\text{limit}+1)}$ 
// using binary exponentiation and polynomial multiplication (via conv).
vector<int> poly_pow(vector<int> poly, int p, int limit = 1e9) {
    vector<int> ans{1}; // Start with the multiplicative identity polynomial: [1]
    while (p) {
        if (p & 1) {
            // If the current bit in p is 1, multiply the result by the current

```

```

poly
    ans = conv(ans, poly);
}

// Square the polynomial for the next bit in exponent
poly = conv(poly, poly);

// Resize to keep degree within limit (truncate higher-degree terms)
ans.resize(limit + 1);
poly.resize(limit + 1);

// Move to next bit of exponent
p >>= 1;
}
return ans;
}

```

NTT

```

// Number Theoretic Transform (NTT) Template
// Efficient for polynomial multiplication under modulus

#include <bits/stdc++.h>
using namespace std;

#define ll long long

// NTT-friendly mod, also used as default
const ll mod = (119 << 23) + 1; // = 998244353
const int root = 62; // Primitive root for mod

// Modular exponentiation
ll modpow(ll b, ll e) {
    ll ans = 1;
    while (e) {
        if (e & 1) ans = ans * b % mod;
        b = b * b % mod;
        e >>= 1;
    }
    return ans;
}

// Compute a primitive root of mod (should be of the form 2^k * m + 1)
int generator() {
    vector<int> fact;
    int phi = mod - 1, n = phi;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0) n /= i;
        }
    }
    for (int i = 2; i <= n; ++i) {
        bool ok = true;
        for (int f : fact) {
            if (i % f == 0) {
                ok = false;
                break;
            }
        }
        if (ok) return i;
    }
    return -1;
}

```

```

    }
}
if (n > 1) fact.push_back(n);

for (int res = 2; res <= mod; ++res) {
    bool ok = true;
    for (int f : fact)
        if (modpow(res, phi / f) == 1) {
            ok = false;
            break;
        }
    if (ok) return res;
}
return -1;
}

// General modular exponentiation with arbitrary modulus
int modpow(int b, int e, int m) {
    int ans = 1;
    while (e) {
        if (e & 1) ans = (ll)ans * b % m;
        b = (ll)b * b % m;
        e >>= 1;
    }
    return ans;
}

// In-place Cooley-Tukey NTT (mod must be of the form c*2^k+1)
void ntt(vector<int>& a) {
    int n = a.size(), L = 31 - __builtin_clz(n);

    static vector<int> rt(2, 1);
    static int max_n = 2;

    for (int k = max_n, s = __builtin_ctz(k); k < n; k *= 2, s++) {
        rt.resize(2 * k);
        int z[] = {1, modpow(root, mod >> s, mod)};
        for (int i = k; i < 2 * k; ++i)
            rt[i] = (ll)rt[i / 2] * z[i & 1] % mod;
    }
    max_n = max(max_n, n);

    vector<int> rev(n);
    for (int i = 0; i < n; ++i)
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;

    for (int i = 0; i < n; ++i)
        if (i < rev[i]) swap(a[i], a[rev[i]]);

    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; ++j) {
                int z = (ll)rt[j + k] * a[i + j + k] % mod;
                int& ai = a[i + j];

```

```

        a[i + j + k] = ai - z + (z > ai ? mod : 0);
        ai += (ai + z >= mod ? z - mod : z);
    }
}
}

// Convolution using NTT
vector<int> conv(const vector<int>& a, const vector<int>& b) {
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(s), n = 1 << B;
    int inv = modpow(n, mod - 2, mod);
    vector<int> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    for (int i = 0; i < n; ++i)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

// Chinese Remainder Theorem for 2 moduli
ll CRT(ll a, ll m1, ll b, ll m2) {
    __int128 m = (__int128)m1 * m2;
    ll ans = a * m2 % m * modpow(m2, m1 - 2, m1) % m + m1 * b % m * modpow(m1, m2
- 2, m2) % m;
    return ans % m;
}

// CRT for 3 moduli to desired_mod (like 1e9+7)
int CRT(int a, int b, int c, int m1, int m2, int m3) {
    __int128 M = (__int128)m1 * m2 * m3;
    ll M1 = (ll)m2 * m3;
    ll M2 = (ll)m1 * m3;
    ll M3 = (ll)m1 * m2;

    int M_1 = modpow(M1 % m1, m1 - 2, m1);
    int M_2 = modpow(M2 % m2, m2 - 2, m2);
    int M_3 = modpow(M3 % m3, m3 - 2, m3);

    __int128 ans = (__int128)a * M1 * M_1;
    ans += (__int128)b * M2 * M_2;
    ans += (__int128)c * M3 * M_3;

    return (ans % M) % desired_mod;
}

// Global definitions for CRT mods and root
int mod, root;
int desired_mod = 1e9 + 7;
const int mod1 = 167772161;
const int mod2 = 469762049;
const int mod3 = 754974721;

```



```

const int root1 = 3;
const int root2 = 3;
const int root3 = 11;

void solve() {
    mod = mod1, root = root1;
    auto c1 = conv(a, b);

    mod = mod2, root = root2;
    auto c2 = conv(a, b);

    mod = mod3, root = root3;
    auto c3 = conv(a, b);

    vector<int> result(c1.size());
    for (int i = 0; i < result.size(); ++i)
        result[i] = CRT(c1[i], c2[i], c3[i], mod1, mod2, mod3);
}

```

Examples

```

// Exact match but multiplied with the number of chars
void solve(int tc) {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<int> poly1(n), poly2(m);

    vector<int> ans_match(n);

    for (int i = 0; i < 26; ++i) {
        for (int j = 0; j < n; ++j) {
            poly1[j] = (s[j] - 'a') == i;
        }
        for (int j = 0; j < m; ++j) {
            poly2[j] = (patt[m-j-1] - 'a') == i;
        }
        vector<int> ans = multiply(poly1, poly2);
        for (int j = 0; j < n; ++j) {
            ans_match[j] += ans[m-1+j];
        }
    }

    int tot = 0;
    vector<int> pos;
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');
    for (int i = 0; i < n; ++i) {

```

```

        if(ans_match[i] == m - wild_cnt) {
            ++tot;
            pos.push_back(i);
        }
    }

    cout << tot << "\n";
    for(auto & p : pos) cout << p << " ";
    cout << "\n";
}

```

Example 2 WildCards

```

void solve(int tc) {

    string s, patt; cin >> s >> patt;
    int n = (int)s.length(), m = (int)patt.length();

    vector<cd> poly1(n), poly2(m);

    for (int i = 0; i < n; ++i) {
        double angle = 2*PI*(s[i]-'a')/26;
        poly1[i] = cd(cos(angle), sin(angle));
    }
    for (int i = 0; i < m; ++i) {
        if(patt[m-i-1] == '*') poly2[i] = cd(0,0); // Wild Card
        else {
            double angle = 2*PI*(patt[m-i-1]-'a')/26;
            poly2[i] = cd(cos(angle), -sin(angle));
        }
    }

    vector<cd> ans = multiply(poly1, poly2);
    int wild_cnt = (int)count(patt.begin(), patt.end(), '*');

    int tot = 0;
    vector<int> pos;
    for (int i = 0; i < n; ++i) {
        if(fabs(ans[m-1+i].real() - (m - wild_cnt)) < eps && fabs(ans[m-1+i].imag()) < eps) {
            ++tot;
            pos.push_back(i);
        }
    }

    cout << tot << "\n";
    for(auto & p : pos) cout << p << " ";
    cout << "\n";
}

```

```
}
```

Number of Inversions

```
void solve() {
    string s;
    cin >> s;
    ll n = s.length();
    ll sh = n-1;
    vector<ll> p1(n), p2(n+sh);
    for (ll i = 0; i < n; i++)
        p1[i] = s[i] == 'A';
    for (ll i = 0; i < n; i++)
        p2[sh-i] = s[i] == 'B';

    vector<ll> ans = multiply(p1, p2);

    for (ll i = 1; i <= n-1; i++)
        cout << ans[i+sh] << ln;
}
```

Polynomial Power

```
vector<int> pow(vector<int> a, ll b) {
    vector<int> res = {1};
    while (b > 0) {
        if (b & 1) res = multiply(res, a);
        a = multiply(a, a);
        b >>= 1;
    }
    return res;
}

int32_t main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    #ifndef ONLINE_JUDGE
        freopen("int.txt", "r", stdin);
        freopen("out.txt", "w", stdout);
    #endif

    ll n, k;
```

```

cin >> n >> k;
vector<int> v(1001);
for (ll i = 0; i < n; i++) {
    ll t;
    cin >> t;
    v[t] = 1;
}

vector<int> ans = pow(v, k);
for (ll i = 0; i < ans.size(); i++) {
    if (ans[i] > 0)
        cout << i << ' ';
}
cout << endl;

return 0;
}

```

Smallest Number doesn't divide any

```

const ll N = 2e5;
vector<vector<int>> divis;

void solve() {
    ll n;
    cin >> n;
    vector<ll> v(n);
    for (ll i = 0; i < n; i++) cin >> v[i];

    ll sh = *max_element(v.begin(), v.end());
    vector<ll> p1(sh+1), p2(2*sh+1);
    for (ll i = 0; i < n; i++) {
        p1[v[i]]++;
        p2[sh-v[i]]++;
    }

    vector<ll> ans = multiply(p1, p2);
    vector<ll> d(N+2);
    for (ll i = 1; i <= sh; i++) {
        if (ans[i + sh]) {
            for (ll j = 0; j < divis[i].size(); j++)
                d[divis[i][j]] = 1;
        }
    }

    ll k = 1;
    while (d[k]) k++;
    cout << k << ln;
}

```

```
signed main() {
    divis = vector<vector<int>>(N+2);
    for(int i=1;i<=N;i++)
        for(int j=i;j<=N;j+=i)
            divis[j].push_back(i);

    ll t = 1;
    cin >> t;
    while (t--)
        solve();

    return 0;
}
```

Fast Multiplication

```
void solve() {
    string s1, s2;
    cin >> s1 >> s2;
    vector<ll> p1(s1.size()), p2(s2.size());
    for (ll i = 0; i < s1.size(); i++)
        p1[i] = s1[i] - '0';
    for (ll i = 0; i < s2.size(); i++)
        p2[i] = s2[i] - '0';
    reverse(p1.begin(), p1.end());
    reverse(p2.begin(), p2.end());

    vector<ll> ans = multiply(p1, p2);
    bool flag = false;
    for (ll i = ans.size()-1; i >= 0; i--) {
        if (ans[i] != 0)
            flag = true;
        if (flag)
            cout << ans[i];
    }
    if (!flag)
        cout << 0;
    cout << ln;
}
```

$$(2*B = A + C) \text{ or } (B - A = C - B)$$

```
void solve() {
    ll n;
    cin >> n;
```

```

map<ll, ll> m;
vector<ll> p1(1e6+1);
for (ll i = 0; i < n; i++) {
    ll a; cin >> a;
    m[2*a]++;
    p1[a]++;
}
vector<ll> res = multiply(p1, p1);
ll ans = 0;
for (ll i = 1; i <= 2e6; i++) {
    if (res[i] > 0) {
        res[i] -= (m[i] != 0);
        res[i] /= 2;
        ans += m[i] * res[i];
    }
}
cout << ans << ln;
}

```

Triple Sum

```

void solve() {
    ll n;
    cin >> n;
    ll sh = 2e4;
    vector<ll> in(n);
    vector<ll> v(2 * sh + 1);
    for (ll i = 0; i < n; i++) {
        ll t;
        cin >> t;
        v[sh + t]++;
        in[i] = t;
    }

    vector<ll> ans = multiply(v, multiply(v, v));

    vector<ll> p2(4*sh + 1);
    for (ll i = 0; i < n; i++)
        p2[in[i]*2 + 2*sh] += 1;

    vector<ll> d2 = multiply(p2, v);

    for (ll i = 0; i < d2.size(); i++)
        ans[i] -= 3*d2[i];

    for (ll i = 0; i < n; i++)
        ans[3*in[i] + 3*sh] += 2;

    for (ll i = -6e4; i <= 6e4; i++) {
        ll res = ans[i+3*sh]/6;
    }
}

```

```

        if (res != 0)
            cout << i << " : " << res << ln;
    }

}

```

Problem 1

In other words, you should find the number of segments of a given array such that there are exactly k numbers of this segment which are less than x .

Nikita wants to get answer for this question for each k from 0 to n , where n is the size of the array.

```

void solve() {
    ll n, x;
    cin >> n >> x;
    vector<ll> v(n+1);
    for (ll i = 1; i <= n; i++) {
        cin >> v[i];
        v[i] = (v[i] < x);
    }
    for (ll i = 1; i <= n; i++) v[i] += v[i-1];

    vector<ll> p1(2e5+1);
    for (ll i = 0; i <= n; i++)
        p1[v[i]]++;

    ll sh = 2e5;
    vector<ll> p2(2*sh + 1);
    for (ll i = 0; i <= n; i++)
        p2[sh-v[i]]++;

    vector<ll> ans = multiply(p1, p2);

    for (ll i = 0; i <= 2e5; i++)
        ans[sh] += p1[i] * (p1[i]-1) / 2 - p1[i] * p1[i];

    for (ll i = 0; i <= n; i++)
        cout << ans[i+sh] << ' ';

    cout << ln;
}

```

Effecient NTT

```

#include <bits/stdc++.h>
using namespace std;

#define ll long long

const int MAX_LOG = 20; // Supports sizes up to 2^20
vector<vector<int>>> bitrev(MAX_LOG + 1);
map<int, vector<vector<int>>> mod_roots; // mod → roots table

// Modular exponentiation with arbitrary mod
int modpow(int base, int exp, int m) {
    int res = 1;
    while (exp) {
        if (exp & 1) res = (ll)res * base % m;
        base = (ll)base * base % m;
        exp >>= 1;
    }
    return res;
}

// Precompute bit-reversal tables
void precompute_bitrev() {
    for (int k = 1; k <= MAX_LOG; ++k) {
        int n = 1 << k;
        bitrev[k].resize(n);
        for (int i = 0; i < n; ++i)
            bitrev[k][i] = (bitrev[k >> 1][i >> 1] >> 1) | ((i & 1) << (k - 1));
    }
}

// Precompute roots of unity for given modulus and primitive root
void precompute_roots_for_mod(int mod, int root) {
    vector<vector<int>>> roots(MAX_LOG + 1);
    for (int k = 1; k <= MAX_LOG; ++k) {
        int n = 1 << k;
        roots[k].resize(n);
        int w = modpow(root, (mod - 1) / n, mod);
        roots[k][0] = 1;
        for (int i = 1; i < n; ++i)
            roots[k][i] = (ll)roots[k][i - 1] * w % mod;
    }
    mod_roots[mod] = move(roots);
}

// In-place NTT (forward/inverse)
void ntt(vector<int>& a, bool invert, int mod, const vector<int>& rev, const
vector<int>& roots) {
    int n = a.size();

    // Bit reversal
    for (int i = 0; i < n; ++i)
        if (i < rev[i]) swap(a[i], a[rev[i]]);

```



```

    for (int len = 1, level = 1; len < n; len <= 1, ++level) {
        for (int i = 0; i < n; i += len * 2) {
            for (int j = 0; j < len; ++j) {
                int u = a[i + j];
                int v = (ll)a[i + j + len] * roots[invert ? (1 << level) - j : j]
% mod;

                a[i + j] = u + v < mod ? u + v : u + v - mod;
                a[i + j + len] = u - v >= 0 ? u - v : u - v + mod;
            }
        }
    }

    if (invert) {
        int inv_n = modpow(n, mod - 2, mod);
        for (int& x : a)
            x = (ll)x * inv_n % mod;
    }
}

// Multiply polynomials a and b under given mod/root
vector<int> convolution(vector<int> a, vector<int> b, int mod, int root) {
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(s), n = 1 << B;

    precompute_roots_for_mod(mod, root);
    auto& roots = mod_roots[mod][B];
    auto& rev = bitrev[B];

    a.resize(n), b.resize(n);
    ntt(a, false, mod, rev, roots);
    ntt(b, false, mod, rev, roots);
    for (int i = 0; i < n; ++i)
        a[i] = (ll)a[i] * b[i] % mod;
    ntt(a, true, mod, rev, roots);
    a.resize(s);
    return a;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precompute_bitrev(); // once globally

    // Moduli and roots
    const int mod1 = 998244353, root1 = 3;

    // Sample polynomials
    vector<int> A = {1, 2, 3};
    vector<int> B = {4, 5, 6};

    // Compute C = A * B under mod1

```

```

vector<int> C = convolution(A, B, mod1, root1);

for (int x : C)
    cout << x << ' ';
cout << '\n';
}

```

Matrix Exponent

```

const ll MOD = 1e9+7;
const ll MAT = 103;
struct M {
    ll t[MAT][MAT];
    ll n;
    M(ll n) {
        this->n = n;
        memset(t, 0, sizeof(t));
    }
    M operator* (const M& a) const {
        M c = M(n);
        signed long long mo2 = 4 * MOD * MOD;
        for (ll i = 0; i < n; i++) {
            for (ll j = 0; j < n; j++) {
                for (ll k = 0; k < n; k++) {
                    c.t[i][k] += t[i][j] * a.t[j][k];
                    if (c.t[i][k] > mo2) c.t[i][k] -= mo2;
                }
            }
        }
        for (ll i = 0; i < n; i++) for (ll j = 0; j < n; j++) c.t[i][j] %= MOD;
        return c;
    }
};

M fastPow(ll n, M base, ll power) {
    M ret = M(n);
    for (ll i = 0; i < n; i++) ret.t[i][i] = 1;
    while (power > 0) {
        if (power & 1) ret = ret * base;
        base = base * base;
        power >>= 1;
    }
    return ret;
}

vector<vector<ll>> multiplyMatrices(vector<vector<ll>> &A, vector<vector<ll>> &B)
{
    int m = A.size(), k = A[0].size(), n = B[0].size();
    vector<vector<ll>> result(m, vector<ll>(n, 0));
    for (int i = 0; i < m; ++i) {

```

```
    for (int j = 0; j < n; ++j) {  
        for (int x = 0; x < k; ++x) {  
            result[i][j] += A[i][x] * B[x][j];  
        }  
    }  
    return result;  
}
```